Celeste RL Agent

Bee Dahlgren
California Baptist University, abigailopal.dahlgren@calbaptist.edu
https://github.com/burmblebee/Celeste-RL-Agent

High-precision platformers present challenging problems for reinforcement learning (RL) due to sparse rewards, unforgiving failure states, and stringent timing constraints that require precise control. This work investigates the feasibility of training an RL agent to navigate *Celeste*, a critically acclaimed 2D platformer known for its precision and complex movement dynamics. To enable experimentation, GymBridge was created, a custom Everest mod that exports real-time game state data from *Celeste* to Python, where a Dueling Deep Q-Network (Dueling DQN) learns from structured observations including player kinematics, dash availability, and environmental grids.
Although the agent has not currently achieved stable long-term learning, it successfully completes a test level intermittently, demonstrating partial mastery of the task. The system exposes challenges in connecting RL systems to real, modded game engines: inconsistent state packets, slow iteration cycles, desynchronization between action timing and physics updates, and checkpoint incompatibilities arising from evolving observation structures.
This work contributes (1) the first Gym-style environment for *Celeste*, (2) a cross-process RL pipeline combining modded game state with a Dueling DQN agent, and (3) empirical insights into the limits of value-based RL in high-precision platformers. Results highlight both the promise and challenges of applying RL to environments that require fine-grained motor control.

Keywords: Dueling DQN, Reinforcement Learning, Celeste

1 Introduction

Reinforcement learning has achieved significant breakthroughs in Atari gameplay, continuous control robotics, and large-scale strategic planning. Still, many fundamental challenges remain for RL agents operating in high-precision environments with sparse rewards and rapid failure conditions. *Celeste* exemplifies this difficulty: success depends on sub-frame-accurate movement, complex dash mechanics, timed movement, and tightly choreographed jumps, making it a challenging benchmark for fine-grained decision-making.

This project investigates the following research question: Can a reinforcement learning agent learn to reliably navigate a high-precision *Celeste* level using a custom Gym-compatible interface?

Addressing this question required designing a real-time state extraction tool, engineering a communication protocol between *Celeste* and Python, and implementing a value-based RL agent capable of learning from structured observations.

This work makes four primary contributions:

1. GymBridge, a custom Everest mod that streams detailed *Celeste* game state (position, velocity, dash availability, grounded/wall status, stamina, death state, and local tile grids) to a Python

process.

2. A Python Gym-style environment (CelesteEnv) that processes observations, computes rewards, and interfaces with a Dueling DQN agent.

3. A complete training pipeline, including action serialization, checkpointing, replay buffering, and network updates.

4. Empirical evidence demonstrating partial success: the agent can occasionally complete a test level but learns inconsistently due to unstable observations and environment constraints.

Section 2 reviews related RL and modding literature. Section 3 details the system design, reward structure, and Dueling DQN architecture. Section 4 reports experimental results. Section 5 discusses limitations, lessons learned, and opportunities for future work.

2 Related Work and Background

Deep Q-Networks (DQN), as shown by Mnih et al. (2015), established that deep RL can achieve human-level performance on discrete-action Atari benchmarks. When determining the neural network to use with this project, it was found that vanilla DQN tends to suffer from unstable value estimates and slow adaptation in environments with rapid failure states; both core characteristics of *Celeste*. Double DQN mitigates overestimation bias (Hasselt et al., 2016), but it still relies on a single-stream value representation that learns inefficiently under sparse, delayed rewards. Policy-gradient methods, such as PPO, were also considered. However, PPO's strength lies primarily in continuous-action or stochastic-control problems; its sample inefficiency and sensitivity to reward sparsity make it less suitable for domains where obtaining informative rollouts is costly.

Dueling DQN (Wang et al., 2016) offers a more direct advantage for this task by factorizing state value and action advantages, enabling the agent to identify useful states even when specific actions receive little feedback. This architecture enhances learning stability and accelerates credit assignment under sparse reward conditions. Although Atari domains provided early success, their action demands are coarse compared to those of high-precision platformers. *Celeste*'s mechanics resemble those of robotics control more than Atari: states evolve rapidly, incorrect movements cause immediate failure, and rewards are heavily delayed. Sparse-reward challenges are well-documented (Ng et al., 1999), motivating the reward-shaping strategies adopted here.

2.1 RL in Modded Game Environments

Prior work has developed RL interfaces for existing games through custom bridges, for example, VizDoom (Kempka et al., 2016), Project Malmo for Minecraft (Johnson et al., 2016), and OpenAI Retro. These frameworks demonstrate the feasibility of connecting real-time games to RL systems, but none involve the precision mechanics characteristic of *Celeste* or provide direct access to physical simulation variables.

2.2 *Celeste* Modding and TAS Frameworks

The Everest mod loader enables runtime modification of *Celeste*, allowing for the extraction of physics variables and environmental structure. The CelesteTAS-EverestInterop repository, created by the *Celeste* TAS Team, provides fine-grained control inputs for TAS (tool-assisted speedruns), offering potential for imitation learning-based RL. However, no prior work uses *Celeste* as an RL benchmark or combines TAS features with value-based learning.

3.1 System Architecture

The system consists of two components. The GymBridge Everest mod extracts *Celeste* engine state each frame—player position and velocity, dash availability, stamina, grounded and wall-grab flags, direction, death state, and a local vision grid—and sends this as a JSON packet over TCP. The Python framework then converts the raw game state into normalized observations, computes shaped rewards, and runs a Dueling DQN agent with experience replay. The agent outputs a 5-element action vector (left, right, jump, dash, grab), which GymBridge injects back into *Celeste*. This setup mirrors Gym-style environments while delegating all simulation to the *Celeste* engine.

3.2 Observation Structure

Each observation includes player position and velocity, dash and movement flags, stamina, a 16×16 tile grid encoding solids and hazards, and a death flag, forming a 266-dimensional state vector.

3.3 Action Space

Actions are expressed as a 5-dimensional control vector governing horizontal movement, jumping, dashing, and grabbing. These are interpreted as binary inputs and passed directly into *Celeste's* input system.

3.4 Reward Design

Because *Celeste* provides no incremental rewards, shaping was required. The reward function combines movement incentives, distance-based guidance, and event rewards:

$+0.2 \cdot \Delta x$ for rightward progress; a penalty for sustained backward motion; $+0.15 \cdot (\text{old\_dist} - \text{new\_dist})$ for reducing distance to the exit; +0.05 per frame for survival; –0.3 for dropping more than 40 units below the exit; +25 for entering a checkpoint area; and +75 for reaching the exit, which ends the episode. Distance trackers update each step to maintain consistent gradients toward the goal.

3.5 Dueling DQN Architecture

The agent uses a Dueling DQN designed for *Celeste's* mixed kinematic and grid input. A shared fully connected encoder feeds into value and advantage streams, which recombine to produce Q-values. The output corresponds to 23 discrete joint actions representing meaningful movement combinations (e.g., diagonal dashes, jump–dash sequences). This reduces the otherwise large control space into a manageable discrete set. Training uses Adam with a learning rate of 0.99, soft target updates ($\tau = 0.005$), gradient clipping, Prioritized Replay (buffer size 200k), exponentially decaying ε-greedy exploration, and 3-frame action repeats for smoother control.

3.6 Checkpointing and Persistence

Checkpoint files store network weights, the target network, ε, and the replay buffer. However, changes to

the observation structure between runs often resulted in shape-mismatch errors, requiring the old checkpoints to be discarded.

3.8 Data

Training data is generated online through agent–environment interaction. Each frame yields a normalized 266-dimensional state vector and one of 23 discrete actions. Invalid or incomplete packets are dropped or zero-filled to avoid NaNs. All data is synthetic and contains no personal information, so the system raises no ethical concerns.

3.9 Evaluation Metrics

Evaluation uses episodic return, success rate (proportion of episodes reaching the exit), average steps-to-goal, death rate, and stall rate. Stability is measured by comparing reward curves across random seeds. Qualitative inspection—movement smoothness, appropriate jump/dash usage, and avoidance of corner-sticking—provides additional insight into policy quality.

4 Experiments and Results

4.1 Setup

Experiments were conducted on a Windows 10 machine running *Celeste* v1.4.0.0-fna with the Everest mod loader and the custom GymBridge integration. The agent was implemented in Python 3.13 using PyTorch, and all runs were executed in real time without emulation speed-up. Unless otherwise specified, hyperparameters followed common DQN defaults. This setup allowed the Python agent to receive continuous game-state packets from *Celeste* while issuing control actions back to the engine.

4.2 Results

Despite unstable and occasionally malformed observations, the Dueling DQN agent achieved a surprising level of competence. It was able to complete the test level, though not reliably. Some episodes displayed coherent jump–dash behaviors resembling intentional platforming strategies, while others devolved into random movement or collapsed into near-idle policies. These inconsistencies were reflected in the training curves, which lacked monotonic improvement due to persistent input instability throughout the runs.

Several components of the system worked as intended. The environment successfully streamed real-time game state into Python, reward shaping was applied without issue, and the Dueling DQN architecture trained sufficiently to produce at least some successful level completions. Video recordings captured instances of functional agent behavior, demonstrating that the overall architecture is viable and that an RL agent can meaningfully interact with *Celeste*.

However, several recurring failure modes prevented consistent long-term learning. Mismatched grid sizes led to model crashes when the observation dimensionality shifted mid-training. Random desynchronization between *Celeste's* update loop and the Python listener introduced inconsistent action timing, and several training runs were disrupted by checkpoint corruption that produced NaN outputs upon reload. Together, these issues prevented stable convergence and limited the reliability of the final

policy. Additionally, many *Celeste* attributes are hidden features, making it difficult or impossible to expose them using GymBridge in certain cases.

5 Discussion and Future Work

5.1 Insights

This project demonstrates that while RL agents can achieve nontrivial behavior in high-precision platformers, stable training requires far more engineering support than conventional benchmarks. The *Celeste* engine was not designed for deterministic RL interaction, leading to issues that overshadow typical algorithmic challenges.

5.2 Limitations

Training was heavily constrained by unreliable and malformed state packets, which frequently destabilized episodes. Slow iteration cycles—often over 60 seconds per reload—and the absence of fast-forward capabilities limited experimentation. The observation space changed frequently, requiring regular updates, and the DQN pipeline was highly sensitive to missing or corrupted values. Combined, these issues prevented stable long-horizon learning.

5.3 Future Directions

Future work should prioritize stronger input validation to block malformed packets and adopt fixed-size grid representations for consistent observations. Imitation learning from TAS movement, curriculum-based room progression, and model-predictive rollouts could improve both exploration and control. Replay-based debugging tools would also help diagnose faulty packets more effectively.

5.4 Lessons Learned

This project highlighted the challenges of RL in complex environments with sparse rewards. *Celeste's* precise movement mechanics and long action chains mean the agent receives useful feedback only rarely, making it hard to assign credit and build stable behaviors. Even small observation errors exacerbate this difficulty, leading to inconsistent learning and fragile policies. Overall, the results highlight a central RL challenge: without dense or well-shaped rewards, agents struggle to learn reliably in rich, high-precision environments.

6 Conclusion

This work provides the first Gym-compatible environment for *Celeste*, enabling real-time RL interaction through a custom Everest mod and a Dueling DQN pipeline. While the agent does not yet exhibit reliable mastery, its ability to complete a test level intermittently demonstrates that structured state features and reward shaping can yield meaningful behavior.

The system also reveals the significant engineering challenges inherent in modded-game RL research and sets the stage for future work in high-precision control environments outside traditional RL benchmarks.

References

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. 2015. Human-level control through deep reinforcement learning. Nature 518, 7540, 529–533. https://doi.org/10.1038/nature14236

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. 2016. Dueling network architectures for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning (ICML '16). PMLR, 1995–2003. https://proceedings.mlr.press/v48/wangf16.html

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. arXiv:1606.01540. https://arxiv.org/abs/1606.01540

Michael Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. 2016. VizDoom: A Doom-based AI research platform for visual reinforcement learning. In 2016 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 1–8. https://doi.org/10.1109/CIG.2016.7860433

Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. 2016. The Malmo platform for artificial intelligence experimentation. In Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI '16). AAAI Press, 4246–4247. https://www.ijcai.org/Proceedings/16/Papers/631.pdf

Andrew Y. Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: theory and application to reward shaping. In Proceedings of the 16th International Conference on Machine Learning (ICML '99). Morgan Kaufmann, 278–287. https://dl.acm.org/doi/10.5555/645528.657613

Hado van Hasselt, Arthur Guez, and David Silver. 2015. Deep reinforcement learning with double Q-learning. In Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI '16). AAAI Press, 2094–2100. https://arxiv.org/abs/1509.06461

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347. https://arxiv.org/abs/1707.06347